VULKAN

Explicit GPU programming from first principles

Handbook

How the course works, what you need, how GPUs and Vulkan fit together, and how to set up

1 The course

2 How to work

3 GPUs from the outside in

4 Vulkan and OpenGL

5 Setup

6 Reference library

7 Progress tracker

First edition, October 2026

Contents

1 The course 3
What you will have built 3
Four units 4
Four threads 4
Who the course is for 5
The volumes and the code 5
Conventions 5
2 How to work 6
The anatomy of a chapter 6
Labs and done-when criteria 6
The notebook and the evidence 6
Using the companion code 6
Measuring properly 7
Getting unstuck 7
3 GPUs from the outside in 8
Host and device 8
Designed for throughput 8
SIMT: one instruction, many lanes 9
Hiding latency 9
The memory hierarchy 10
Bandwidth and arithmetic 10
Why explicit APIs 10
4 Vulkan and OpenGL 12
Two histories 12
Two designs 12
One program in both 12
What OpenGL's driver did for you 13
The concepts side by side 14
Costs and benefits 15
When OpenGL is still the right tool 15
5 Setup 16
Hardware 16
Windows 16
Linux 16
macOS 17
Check the installation 17
The companion code 17
Troubleshooting 18
6 Reference library 19
Specifications 19
Guides and samples 19
Synchronisation and render graphs 19
Memory 19
GPU computing 20
Ray tracing 20
Books on Vulkan 20
Vendor guides 20
Tools 20
7 Progress tracker 21

Vulkan, First edition, October 2026. © 2026 O'Neill Group LLC. Vulkan and the Vulkan logo are registered trademarks of the Khronos Group Inc. OpenGL and the oval logo are trademarks or registered trademarks of Hewlett Packard Enterprise in the United States and other countries, used by permission by Khronos. All other trademarks belong to their owners. This course is not endorsed by the Khronos Group.

PART 1

The course

This course teaches GPU programming with Vulkan from first principles, to programmers who know C++ well and have never used Vulkan. It treats the GPU as a separate computer, with its own memory, its own queues of work and its own timeline, and Vulkan as the explicit interface to it. Compute comes first, because compute shaders show the GPU's execution model without the machinery of rendering, and synchronisation gets a unit of its own, because it is where most Vulkan programs go wrong. Graphics follows, built on everything before it. Throughout, the course compares Vulkan with OpenGL, so that readers who know OpenGL can map what they know and readers who do not can see why Vulkan is shaped the way it is.

Every chapter has labs with measurable done-when criteria. Every listing in these volumes is taken from companion code that compiles, runs and checks its own results, and every console listing is real output from those programs. All of it was run on an Apple M2 Pro through MoltenVK with the validation layers on, and the text says wherever other hardware behaves differently.

What you will have built

- **A Vulkan compute program with nothing hidden.** SAXPY written against the raw C API, from the instance to the dispatch and the check of every element, and six broken versions of it diagnosed from their validation messages.
- **A library of parallel primitives.** Reductions, prefix sums, stream compaction, histograms and a radix sort, each measured against your GPU's memory bandwidth, and image filters, an N-body simulation and a tuning harness built on them.
- **A ray tracer in a compute shader.** A Monte Carlo path tracer with spheres, triangle meshes and a bounding volume hierarchy, diffuse, metallic and glass materials, and progressive accumulation, verified against a CPU reference and measured in rays per second.
- **A synchronisation toolkit.** A catalogue of hazards and the barriers that prevent them, each checked by synchronisation validation; host and device pipelines on timeline semaphores; async compute with queue ownership transfers; frames in flight; and a render graph that derives every barrier and layout transition, aliases transient memory, and schedules work across queues.
- **A renderer.** The graphics pipeline from a triangle to a textured, depth-tested mesh, presentation through a swapchain, and a million particles simulated in compute and culled on the GPU.
- **A port from OpenGL.** The same scene in OpenGL and in Vulkan, with the CPU cost of each measured.
- **A GPU-driven sandbox.** The capstone: simulation, culling, indirect drawing, async compute and frames in flight, organised by your render graph and timed pass by pass, with synchronisation validation silent.

Four units

Unit	Chapters	Hours
1 The Explicit API	Instances, devices and queues; memory, buffers and images; commands and submission; shaders, SPIR-V and pipelines; descriptors and your first dispatch; validation, debugging and a helper layer	32
2 Compute Shaders	The compute execution model; memory in shaders; parallel patterns; images and image processing; measuring and tuning; dispatch at scale; a ray tracer in a compute shader	44
3 Synchronisation	Why synchronisation is your job; the dependency model; pipeline barriers in practice; fences, semaphores and events; queues and async compute; frames in flight; finding synchronisation bugs; render graphs	45
4 Rendering and the Whole Frame	The graphics pipeline; presentation; drawing with data; compute meets graphics; from OpenGL to Vulkan; scaling up; capstone: a GPU-driven sandbox	47

The hours are study time: reading, labs, problem sets and the checklist. The whole course is about 170 hours; at eight to ten hours a week it takes about five months. The hours are a guide. A lab is finished only when its done-when criteria hold.

Four threads

Four threads run through the units, and most chapters advance more than one.

- **The execution model.** How the GPU runs work: queues and command buffers (Chapters 1.3 and 3.5), invocations, workgroups and subgroups (2.1), memory in shaders (2.2), occupancy and bandwidth (2.5), and the graphics pipeline as a sequence of programmable and fixed stages (4.1).
- **Synchronisation.** The first barrier (1.3), barriers between dispatches (1.5 and Unit 2), the dependency model (3.2), every primitive Vulkan offers (3.3 to 3.6), the tools that find mistakes (1.6 and 3.7), and finally a render graph that writes the barriers for you (3.8), which the capstone uses (4.7).



Highlighted: the projects that close Units 2 to 4, a ray tracer, a render graph and a GPU-driven sandbox.

Figure 1.1. The course map. Each unit builds on the units before it, and Units 2 to 4 each close with a project that draws the unit together.

- **OpenGL.** Every chapter has “Coming from OpenGL” notes that map its ideas to OpenGL’s. Part 4 of this Handbook compares the two APIs as a whole, and Chapter 4.5 ports a program and measures the difference.
- **Measurement.** Timestamps from Unit 2 on, bandwidth and the roofline (2.5), overlap on queues (3.5), frame pacing (3.6), CPU recording costs (4.5 and 4.6), and the per-pass timings of the capstone.

Who the course is for

The course assumes that you write C++ fluently: classes and RAII, move semantics, lambdas and templates at the level of the standard library, and the build tools of your platform. It uses C++20, mostly for designated initialisers, which make Vulkan’s structures readable. It assumes the linear algebra of vectors and 4×4 matrices that 3D graphics uses, which Unit 4 and the ray tracer need, and nothing more advanced.

It assumes no knowledge of Vulkan, GPUs or graphics. Readers who know OpenGL, CUDA or another GPU API will move faster through the early chapters and should use the skip tests at the start of each chapter to decide what to skim.

The volumes and the code

The course is five volumes: this Handbook and four units. The companion code is a single CMake project with one directory per lab project, `code/u1/devices` to `code/u4/sandbox`, and a small helper library, `vkf`, that Unit 1 builds up to and the other units use. It is a separate free download beside the volumes, and Part 5 explains how to build it.

Conventions

- **Versions.** The course targets Vulkan 1.3 core. It uses synchronization2 and dynamic rendering, both core in 1.3, and avoids extensions unless a chapter is about one. Shaders are GLSL 4.60 compiled to SPIR-V 1.6.
- **Spelling.** British spelling in prose, API names verbatim: synchronisation in the text, `synchronization2` in code.
- **Numbering.** Chapters are numbered unit.chapter, so Chapter 3.2 is the second chapter of Unit 3. Labs are numbered unit.chapter.lab, and figures and tables unit.chapter.number.
- **Listings.** Each listing names the file it comes from, relative to the code folder, and shows a region marked in that file. Console listings begin with the command that produced them, run from the code folder. Long validation messages are wrapped to fit the page, and the values of object handles, which change from run to run, are shown as `0x…`.
- **Time** is in hours. Measurements name the device they were made on.
- **Platform.** “The M2 Pro” is the Apple M2 Pro on which this course’s code was run. Commands assume a POSIX shell; on Windows, use the equivalent in PowerShell or a Developer Command Prompt.

PART 2

How to work

The anatomy of a chapter

Every chapter has the same parts, in the same order.

- **The header** gives the study time, what you build, and what the chapter needs: earlier chapters, tools, hardware.
- **Why it matters** says what the chapter is for, in the course and in real GPU work.
- **The skip test** is four or five questions. If you can answer them all without looking anything up, skim the core ideas and do the labs the test names. If any is hard, work through the whole chapter.
- **Core ideas** teaches the material, in order, from first principles. Listings come from the companion code, and console listings from real runs. Callouts mark what to watch for: *Key idea* for the central point, *Pitfall* and *Hazard* for mistakes that are easy to make, and *Coming from OpenGL* for the comparison.
- **Reading** lists the primary sources: sections of the specification, the Vulkan Guide, papers and vendor documents. Read them while you work through the chapter.
- **Labs** each have a goal, a time, the code they start from, numbered steps, *done-when* criteria and the evidence to keep.
- **The problem set** tests understanding without a computer.
- **The completion checklist** lists what you should now be able to do. Tick it honestly.

Each unit ends with a sign-off page: a table of its chapters and a short unit review, a set of tasks to do from memory.

Labs and done-when criteria

A lab is finished when every done-when criterion holds. The criteria are measurable on purpose: a program passes its own check with validation on, a number falls within a range, a message has been recorded, an explanation fits in two sentences. If you cannot meet one, the lab has found something you do not yet understand.

The times are what the lab takes a careful engineer who has done the reading. Some labs will take longer on your machine, especially the first time a tool misbehaves, so budget for it.

The notebook and the evidence

Keep a notebook, on paper or in plain text files in a repository: for every lab, the date, what you did, what you measured, what surprised you and the evidence each lab asks for. Keep your code in version control with one commit per lab, so that you can return to any state.

The notebook is how you notice that a measurement does not match the last one, and how you explain a result months later. It is also the record that the sign-off pages refer to.

Using the companion code

The companion code is complete: every project builds and passes its own checks. Most labs ask you to change it, extend it or break it on purpose. Work in a copy, and when a lab asks you to write something new, write it yourself before you look at any reference. When you compare afterwards, look for differences in approach as well as in output.

The code follows a few rules you can rely on:

- **Every program checks itself.** It verifies its results against a CPU reference or an invariant, prints a result line, and exits with a non-zero status on failure.
- **Validation is on by default.** Programs run with the validation layer and synchronisation validation enabled, and fail if the layer reports an error. If the layer is not installed, they say so and run without it. Set `VKF_VALIDATION=0` only to measure performance.
- **Options are explicit.** Programs take options such as `--n 1000000` or `--out image.png`, and their defaults run in a few seconds.
- **Comments are rare.** The volumes explain the code; a comment in the code states only something the volumes cannot, such as a gotcha or a constraint.

Measuring properly

Many labs ask for timings. GPUs are hard to time well, and a careless measurement is worse than none, because it is easy to mistake for a correct one.

- **Turn validation off.** The validation layers slow every API call, and synchronisation validation slows submission considerably. Every timing in the course is taken with `VKF_VALIDATION=0`.
- **Time the GPU on the GPU.** Wall-clock time around a submission includes the driver, the operating system and the wait. Timestamp queries, introduced in Chapter 2.5, measure the work on the device itself.
- **Warm up, repeat, and report the median.** The first run of anything includes compilation, page faults and clock ramp-up. Run several times and report the median, with the spread.
- **Know your clocks.** GPUs change clock speed with load and temperature, and laptops change it with power source. Measure on mains power, with the machine otherwise idle, and say so.
- **Name the machine.** Record the GPU, the driver version and the operating system with every number.

Getting unstuck

When something goes wrong, work through these in order.

1. **Read the validation message.** Find the VUID it cites in the specification, and read the rule in context. Most problems end here; Chapter 1.6 shows how.
2. **Make it smaller.** Remove everything that does not affect the failure until what remains fits on a screen. The cause is then usually obvious, and if it is not, the small program makes a good question to ask someone.
3. **Look at the GPU's state.** Capture the program with RenderDoc, or with Xcode on macOS, and inspect the buffers and images before and after the failing command. Print from the shader with debug printf.
4. **Check the specification, then the Vulkan Guide.** The specification is long but precise, and its valid-usage lists answer most questions about what is allowed.
5. **Compare with the companion code.** The reference version of each project shows one correct way to do it.

When a program behaves differently on two machines, suspect undefined behaviour first: a missing barrier, an uninitialised image, an out-of-bounds access. Conformant drivers run a valid program the same way, apart from documented differences such as floating-point precision. An invalid program has undefined behaviour, and each driver may run it differently.

PART 3

GPUs from the outside in

This part is a short tour of the hardware that Vulkan programs. It contains no Vulkan, only the ideas the units rely on. Unit 2 returns to each of them with measurements on your own GPU.

Host and device

A GPU is a separate processor, and the most useful way to think of it is as a separate computer. A program runs on the *host*, the CPU, and sends work to the *device*, the GPU. A discrete GPU has its own memory, called VRAM, and reaches the host's memory across the PCI Express bus, which moves about 32 GB/s in each direction at PCIe 4.0 with sixteen lanes, and twice that at PCIe 5.0. An integrated GPU, such as Apple's or those built into most laptop processors, shares memory with the CPU instead. Figure 3.1 shows the parts that matter to a programmer.

The host does not call functions on the GPU. It writes commands into memory and tells the GPU where they are. The GPU's *command processor* reads them and starts the work, at its own pace, while the host carries on. Vulkan exposes this arrangement directly: command buffers hold the commands, queues are where you submit them, and fences and semaphores report when they are done. Unit 1 builds each of these.

Designed for throughput

A CPU core is built to finish one thread's work as soon as possible. It spends most of its silicon on large caches, branch prediction and out-of-order execution, so that a single instruction stream rarely waits. A GPU is built to finish a large amount of work as soon as possible, and accepts that any single piece of it may wait. It spends its silicon on arithmetic units and on keeping a large number of threads ready to run. A large desktop GPU holds on the order of a hundred thousand threads in flight at once.

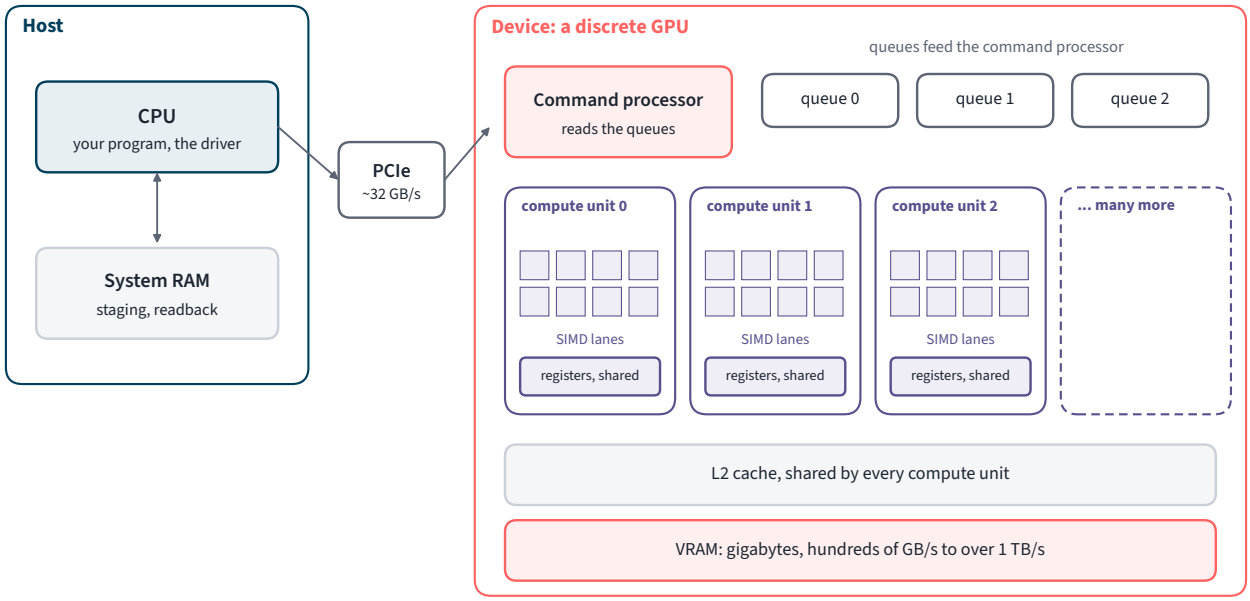


Figure 3.1. Host and device. The host sends work through queues that the GPU's command processor reads; the compute units do the work; every unit shares the L2 cache and the device's memory.

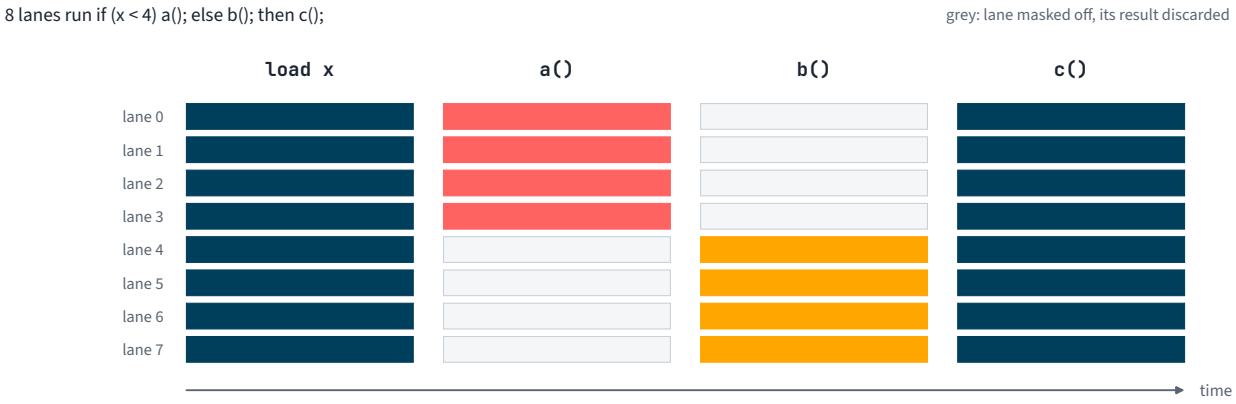


Figure 3.2. Divergence. Eight lanes of a subgroup run an `if` whose condition differs between lanes: the subgroup runs both branches in turn, with the lanes that did not take each branch masked off.

The work is divided among *compute units*: NVIDIA calls them streaming multiprocessors, AMD compute units, Intel Xe-cores, and Apple GPU cores. A desktop GPU has from a few dozen to over a hundred. Each has its own schedulers, arithmetic units, a large register file and a small, fast memory that its threads can share.

SIMT: one instruction, many lanes

Threads on a GPU do not each have their own instruction stream. They are grouped, and every thread in a group executes the same instruction at the same time on its own data: *single instruction, multiple threads*, or SIMT. Vulkan calls the threads *invocations* and the hardware group a *subgroup*. Vendors use their own names and sizes: NVIDIA's *warps* have 32 threads; AMD's *wavefronts* have 64 on older GCN hardware and 32 or 64 on RDNA; Intel uses 8, 16 or 32; Apple's *SIMD-groups* have 32.

When the invocations of a subgroup take different branches, the subgroup executes both branches, one after the other, with the invocations that did not take each branch switched off (Figure 3.2). Code whose branches depend on data can therefore run at a fraction of the hardware's speed. Chapter 2.4 measures it, and the ray tracer in Chapter 2.7 meets it at every bounce.

Hiding latency

Reading memory takes a GPU hundreds of cycles. To hide that time, each compute unit keeps many subgroups resident at once, and when one stalls on a load, the scheduler switches to another that is ready, at no cost (Figure 3.3). As long as some subgroup is ready, the arithmetic units never wait.

How many subgroups a compute unit can hold is called its *occupancy*. It is limited by the resources each subgroup needs: registers, which every invocation uses, and shared memory, which a workgroup uses. A shader that uses many registers leaves room for fewer subgroups, and so has fewer to switch between.

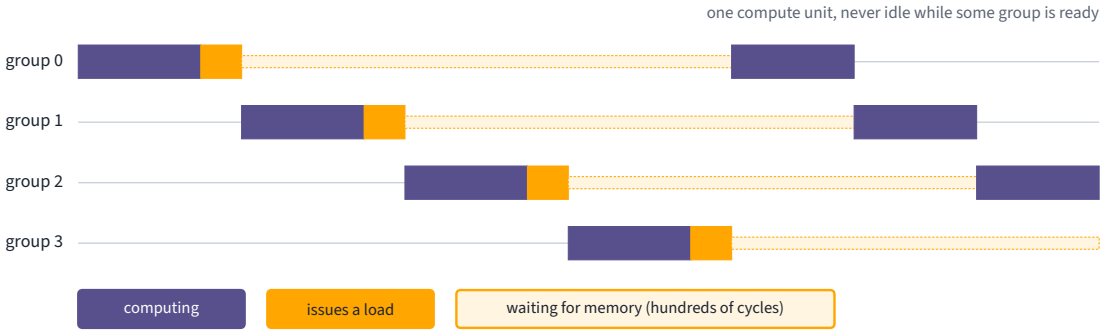


Figure 3.3. Latency hiding. Four subgroups share one compute unit; while each waits for memory, the others compute.

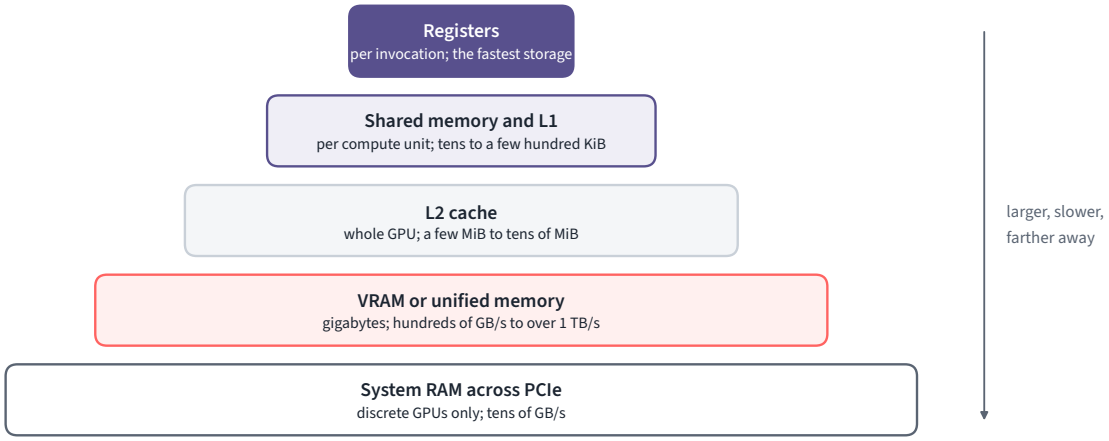


Figure 3.4. The memory hierarchy, from registers to system RAM. Each level is larger, slower and farther from the arithmetic than the one above.

Too little occupancy leaves the compute unit idle while it waits for memory, but more occupancy does not always make a shader faster. Chapter 2.5 measures the trade-off.

The memory hierarchy

GPU memory forms a hierarchy (Figure 3.4). *Registers* are private to each invocation and fastest; a compute unit has hundreds of kilobytes of them, more than its cache. *Shared memory*, a small scratchpad that the invocations of one workgroup share, is almost as fast and is managed by the program. The *L1* and *L2* caches are managed by the hardware; L2 is shared by the whole GPU. *VRAM* holds everything else.

Two properties of the hierarchy shape GPU code. First, memory moves in blocks. When the invocations of a subgroup read neighbouring addresses, the hardware combines their reads into a few wide transactions, which is called *coalescing*. When they read scattered addresses, each read costs a transaction. Second, shared memory lets a workgroup load data once and use it many times, which is the basis of most fast GPU algorithms. Unit 2 relies on both.

Bandwidth and arithmetic

A GPU's arithmetic is far faster than its memory. A desktop GPU can perform tens of trillions of floating-point operations per second but read only about a trillion bytes per second from VRAM, so a kernel must do tens of operations for every byte it reads to keep the arithmetic busy. The ratio of operations to bytes is the kernel's *arithmetic intensity*.

The *roofline model* turns this into a picture (Figure 3.5). A kernel's attainable speed is the lower of two limits: the peak arithmetic rate, and the memory bandwidth multiplied by its arithmetic intensity. The two meet at the *ridge point*. Kernels to its left are *memory-bound*: the only way to make them faster is to move fewer bytes. Kernels to its right are *compute-bound*. SAXPY, the first program of the course, performs two operations for every twelve bytes it moves, and is about as memory-bound as a kernel can be.

Why explicit APIs

OpenGL and Direct3D 11 were designed when a GPU was a fixed-function device driven by a single CPU thread. As GPUs became general processors and CPUs gained cores instead of speed, the drivers behind those APIs became the bottleneck. A driver had to track the state of every object, validate every call, decide when to compile shaders, find dependencies between commands and insert barriers, and manage memory, all on the one thread that owned the context. Much of that work was invisible to the program, and its cost was unpredictable: a draw call could trigger a shader compilation or a copy of a texture.

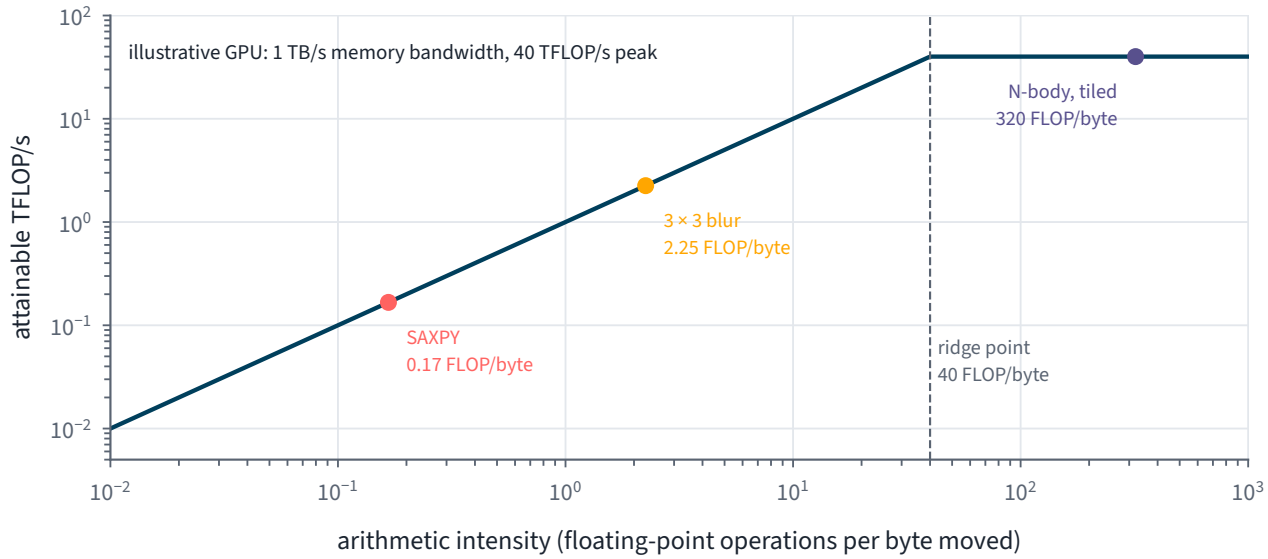


Figure 3.5. The roofline, computed for an illustrative GPU with 1 TB/s of memory bandwidth and 40 TFLOP/s of arithmetic. The kernels are placed by their arithmetic intensity, assuming each byte moves between memory and the GPU once. Chapter 2.5 measures the roofline of your own GPU.

Game consoles had long offered thin, explicit interfaces to their GPUs, and their games were faster for it. AMD's Mantle, announced in 2013, brought the idea to the PC. Apple released Metal in 2014 and Microsoft Direct3D 12 in 2015. Khronos, which maintains OpenGL, built Vulkan from Mantle and released it in 2016. All of these APIs make the same trade. The program takes over work that the driver used to do, and in exchange gets lower and predictable CPU cost, recording on many threads, and control over memory and synchronisation. Part 4 compares Vulkan and OpenGL in detail.

PART 4

Vulkan and OpenGL

Most readers of this course will have met OpenGL, or will meet it in code they maintain. This part compares the two APIs as a whole: where they came from, how they differ in design, what OpenGL's driver did that Vulkan hands to you, and when OpenGL is still the better choice. Each later chapter adds detailed "Coming from OpenGL" notes, and Chapter 4.5 ports a program from one to the other and measures the difference.

Two histories

OpenGL 1.0 was published in 1992 by Silicon Graphics, derived from its proprietary IRIS GL, and for twenty-five years it was the portable way to program graphics hardware. An Architecture Review Board of vendors evolved it until 2006, when the Khronos Group took it over. Each version tracked the hardware of its day: programmable shaders in GLSL with OpenGL 2.0 in 2004; a core profile that dropped the fixed-function pipeline with 3.2 in 2009; compute shaders and shader storage buffers with 4.3 in 2012; direct state access with 4.5 in 2014; and SPIR-V shaders with 4.6 in 2017, the last version. OpenGL ES, a subset for embedded and mobile devices, and WebGL, its browser counterpart, extended its reach. Apple stopped at OpenGL 4.1 and deprecated it in 2018.

Vulkan 1.0 was released by Khronos in February 2016, built from AMD's Mantle with the participation of every major GPU vendor. Its versions have folded widely used extensions into the core: 1.1 in 2018 (subgroups, multiple GPUs), 1.2 in 2020 (timeline semaphores, descriptor indexing, buffer device addresses), 1.3 in 2022 (synchronization2, dynamic rendering) and 1.4 in 2024. Unlike OpenGL, Vulkan has a single API for desktop and mobile, with optional features that each device reports.

Two designs

OpenGL is a *state machine* attached to an implicit *context*. A thread makes a context current, and from then on every call reads or changes that context's global state: the bound buffer, the active texture unit, the current program, the blend mode. Objects are named by integers, and most are edited by binding them first ("bind to edit"). A draw call takes its inputs from whatever state happens to be current, so the driver must check that state at every draw, and only then can it work out what the GPU must do.

Vulkan has no current context and no global state. Every call names the objects it uses. State that a draw or dispatch depends on is collected into objects created ahead of time, above all the *pipeline*, which bakes shaders and fixed-function state together, and the *descriptor set*, which collects resources. Work is recorded into command buffers on any thread and submitted explicitly. The driver does no validation at run time, and the validation layer checks every call during development.

One program in both

Chapter 1.5 writes SAXPY, `y = a·x + y`, in Vulkan in about two hundred lines. The listing below is the host side of the same program in OpenGL 4.5, with context creation and error checks left out. Its shader is the SAXPY shader of Chapter 1.5 with two changes: the buffers are named by `binding` alone, without a `set`, and `a` and `n` are plain `uniform` variables instead of push constants.



Figure 4.1. What the driver does. An OpenGL driver tracks state, validates, compiles, manages memory, finds hazards and decides when to submit. A Vulkan driver translates what the program has already decided; validation moves into a layer used during development.

```
GLuint shader = glCreateShader(GL_COMPUTE_SHADER);
glShaderSource(shader, 1, &source, nullptr);
glCompileShader(shader);
GLuint program = glCreateProgram();
glAttachShader(program, shader);
glLinkProgram(program);

GLuint buffers[2];
glCreateBuffers(2, buffers);
glNamedBufferStorage(buffers[0], bytes, x.data(), 0);
glNamedBufferStorage(buffers[1], bytes, y.data(), 0);

glUseProgram(program);
glUniform1f(glGetUniformLocation(program, "a"), a);
glUniform1ui(glGetUniformLocation(program, "n"), n);
glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 0, buffers[0]);
glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 1, buffers[1]);
for (int r = 0; r < 10; ++r) {
    if (r > 0) glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT);
    glDispatchCompute((n + 255) / 256, 1, 1);
}
glMemoryBarrier(GL_BUFFER_UPDATE_BARRIER_BIT);
glGetNamedBufferSubData(buffers[1], 0, bytes, y.data());
```

Every line has a Vulkan counterpart, and most have several. The context hides the instance, the device and its queue (Chapter 1.1). `glNamedBufferStorage` hides the choice of memory type, the allocation and the binding (1.2). `glCompileShader` and `glLinkProgram` compile at run time what Vulkan compiles to SPIR-V at build time and into a pipeline at start-up, with its layout (1.4). `glBindBufferBase` and `glUniform*` are the descriptor set and the push constants (1.5). The dispatches go into a command buffer that the driver manages and submits for you, and `glGetNamedBufferSubData` waits for them, which in Vulkan is a submission, a barrier to the host and a fence (1.3). Only the barriers between dispatches appear in both programs, because OpenGL's driver cannot see what a shader writes.

What OpenGL's driver did for you

Each of these is something you will do yourself in Vulkan, and each has a chapter.

- **Hazard tracking.** An OpenGL driver knows which command wrote a buffer or texture and which command reads it next, and it inserts the necessary waits and cache operations. It cannot see which

addresses a shader will write, which is why OpenGL 4.2 added `glMemoryBarrier` for shader writes. In Vulkan you record every barrier (Chapter 1.3 and Unit 3), or let a render graph derive them (Chapter 3.8).

- **Memory management.** `glBufferData` asks for storage and the driver chooses where it lives, moves it when it guesses wrong, and keeps old copies alive while the GPU still uses them. In Vulkan you choose memory types, allocate, sub-allocate and decide when memory may be reused (Chapters 1.2 and 4.6).
- **Shader compilation.** An OpenGL driver compiles GLSL at run time with its own compiler, and may compile again when state that the program object does not capture changes, causing stalls at unpredictable draw calls. Vulkan compiles SPIR-V when you create a pipeline, with everything it depends on known (Chapter 1.4).
- **State validation.** OpenGL checks every call, as part of the specification, in a finished program as much as in development, unless the program asks for a context without error checking, which OpenGL 4.6 allows. A Vulkan driver does no validation at run time, and the validation layer checks every call during development (Chapter 1.6).
- **Threading.** An OpenGL context belongs to one thread at a time, so work is submitted from a single thread. Sharing objects between contexts on several threads is possible but fragile. Vulkan command buffers can be recorded on any number of threads, each with its own pool, and submitted from one (Chapter 4.6).
- **Submission and pacing.** An OpenGL driver buffers commands and decides when to send them, and the swap of the default framebuffer hides the synchronisation between frames. In Vulkan you submit, pace frames in flight and synchronise presentation yourself (Chapters 3.6 and 4.2).

The concepts side by side

Table 4.1 maps the OpenGL concepts this course meets to their Vulkan counterparts and to the chapter that teaches each.

OpenGL	Vulkan	Chapter
Context, made current on a thread	`VkInstance`, `VkDevice` and `VkQueue`, named in every call	1.1
`glGetIntegerv` limits; every extension available	Physical device properties, limits and features; extensions enabled at creation	1.1
`glGetError`, the `KHR_debug` callback	The validation layer and `VK_EXT_debug_utils`	1.6
`glGenBuffers` and `glBufferData` or `glBufferStorage`	`vkCreateBuffer`, `vkAllocateMemory`, `vkBindBufferMemory`	1.2
Persistent, coherent `glMapBufferRange`	`vkMapMemory` on `HOST_VISIBLE` and `HOST_COHERENT` memory	1.2
`glTexStorage2D` and `glTexSubImage2D`	`vkCreateImage`, memory, a staging buffer, `vkCmdCopyBufferToImage` and layout transitions	1.2, 4.3
Texture parameters and sampler objects	`VkSampler` and `VkImageView`	2.4, 4.3
GLSL source, `glCompileShader`, `glLinkProgram`	SPIR-V compiled offline, `VkShaderModule`, `VkPipeline`	1.4
`glUniform*`	Push constants, or uniform buffers through descriptors	1.4, 1.5
`glBindBufferBase`, `glBindTexture`, image units	Descriptor sets and `vkCmdBindDescriptorSets`	1.5
`glDispatchCompute`	`vkCmdDispatch`	1.5
`glMemoryBarrier`	`vkCmdPipelineBarrier2` with stages and access masks	1.3, 3.2
`glFenceSync` and `glClientWaitSync`	`VkFence`, timeline semaphores	1.3, 3.4
`glFlush` and `glFinish`	`vkQueueSubmit2`, `vkQueueWaitIdle`	1.3
`glGetProgramBinary`	`VkPipelineCache`	1.4
`glQueryCounter` with `GL_TIMESTAMP`	`VkQueryPool` and `vkCmdWriteTimestamp2`	2.5
Vertex array objects, `glVertexAttribPointer`	Vertex input state in the pipeline, `vkCmdBindVertexBuffers`	4.1
`glEnable(GL_DEPTH_TEST)`, `glBlendFunc`, `glViewport`	Pipeline state, and dynamic state such as `vkCmdSetViewport`	4.1
Framebuffer objects and `glClear`	Dynamic rendering: `vkCmdBeginRendering` with load and store operations	4.1
`glDrawArrays`, `glDrawElements`, indirect draws	`vkCmdDraw`, `vkCmdDrawIndexed`, `vkCmdDrawIndexedIndirect`	4.1, 4.4
`glReadPixels`	`vkCmdCopyImageToBuffer`, a fence, and a mapped read	4.1

OpenGL	Vulkan	Chapter
The default framebuffer and `SwapBuffers`	`VkSurfaceKHR`, `VkSwapchainKHR`, acquire and present	4.2
Shared contexts on several threads	A command pool per thread, secondary command buffers	4.6
Hazards tracked by the driver	Barriers you record, or a render graph that derives them	3.2, 3.8

Table 4.1. OpenGL concepts and their Vulkan counterparts

Costs and benefits

A Vulkan program is several times longer than its OpenGL equivalent: SAXPY is about two hundred lines in Unit 1 against a few dozen in OpenGL. Mistakes that OpenGL would have reported or silently fixed become undefined behaviour, which is why this course runs validation everywhere. A program must decide its pipelines, layouts and memory up front, which takes design, and there is more to learn before the first result.

In return, the CPU cost of each draw and dispatch is low and predictable, because nothing is compiled, copied or synchronised unless the program asks for it. Recording scales across threads. Memory can be sub-allocated, aliased and placed where the program chooses. Explicit synchronisation lets independent work overlap, including on separate queues, which OpenGL cannot express. One API covers desktop and mobile GPUs, with SPIR-V and the validation layers shared by all of them. Because the program does the work the driver used to do, its author can see, measure and fix that work.

When OpenGL is still the right tool

OpenGL remains a reasonable choice for small tools and prototypes whose CPU cost does not matter; for large OpenGL code bases that work and have no performance problem; and for hardware or platforms without a Vulkan driver. On macOS it is frozen at version 4.1, deprecated, and without compute shaders.

Between the two lie libraries that offer explicit concepts with more safety. WebGPU, the W3C standard for the web, also has native implementations such as Google's Dawn and the Rust library wgpu. SDL 3 includes a portable GPU API. OpenGL itself increasingly runs on Vulkan: Mesa's Zink implements OpenGL on top of Vulkan, and Google's ANGLE implements OpenGL ES on Vulkan, Metal and Direct3D. Each of these libraries does internally the work that this course teaches you to do in Vulkan.

PART 5

Setup

The course needs five things: a GPU driver with Vulkan 1.3, the Vulkan SDK's loader, layers and shader tools, a C++20 compiler, CMake with Ninja, and, for Unit 4, GLFW. This part sets them up on Windows, Linux and macOS, builds the companion code and checks it with `vkf_selftest`. Allow two hours.

Hardware

Any GPU whose driver reports Vulkan 1.3 will do. As a rough guide that means NVIDIA GeForce 900 series and later, AMD Radeon RX 400 series and later, Intel graphics from 6th-generation Core processors on, and every Apple Silicon Mac through MoltenVK; Lab 1.1.1 checks your own. A discrete GPU makes the performance chapters more instructive, because its memory and bus behave differently from the CPU's, but every lab works on integrated graphics. The course's code was run on an Apple M2 Pro, and its console listings come from that machine.

Windows

1. **Driver.** Install the current driver from NVIDIA, AMD or Intel. The drivers that Windows Update installs sometimes lack Vulkan.
2. **Vulkan SDK.** Install the current SDK from LunarG at vulkan.lunarg.com. It includes the loader, the validation layers, `glslc`, the SPIR-V tools, `vulkaninfo` and the Vulkan Configurator, and sets the `VULKAN_SDK` environment variable that CMake uses to find them.
3. **Compiler and build tools.** Install Visual Studio 2022 with the "Desktop development with C++" workload, which includes MSVC, CMake and Ninja. Build from an "x64 Native Tools Command Prompt" so that they are on the path.
4. **GLFW,** for Unit 4. The simplest route is vcpkg: `vcpkg install glfw3 opengl-registry`, then add `-DCMAKE_TOOLCHAIN_FILE=<vcpkg>/scripts/buildsystems/vcpkg.cmake` when you configure the code below. The second package supplies Khronos' OpenGL headers, which only Chapter 4.5's port needs.
5. **RenderDoc,** from renderdoc.org.

Linux

1. **Driver.** Mesa provides Vulkan for AMD (RADV) and Intel (ANV); keep it current, since newer versions add features and fix bugs. For NVIDIA, install the proprietary driver.
2. **Vulkan SDK.** Download the Linux tarball from LunarG, extract it, and run `source <sdk>/setup-env.sh` in each shell, or add that line to your shell's start-up file. Distribution packages are an alternative; on Ubuntu 24.04, `sudo apt install libvulkan-dev vulkan-tools vulkan-validationlayers glslc spirv-tools glslang-tools`. They may lag behind the SDK, as explained under "Validation layer versions" below.
3. **Compiler and build tools.** GCC 13 or Clang 17 or later, CMake 3.24 or later, and Ninja: `sudo apt install g++ cmake ninja-build`.
4. **GLFW,** for Unit 4: `sudo apt install libglfw3-dev libgl-dev`. The second package supplies the OpenGL headers that Chapter 4.5's port needs.
5. **RenderDoc,** from your distribution or renderdoc.org.

macOS

Apple's GPUs have no native Vulkan driver. MoltenVK implements Vulkan on top of Metal, as Chapter 1.1 explains, and the rest of the toolchain is the same as elsewhere.

1. **Compiler.** Install the Xcode Command Line Tools with `xcode-select --install`. Install Xcode itself as well if you want to capture GPU work (Chapter 1.6).
2. **Vulkan SDK, recommended.** Install the macOS SDK from LunarG, which includes MoltenVK, the loader, the layers and the tools. Run `source ~/VulkanSDK/<version>/setup-env.sh` in each shell, or add it to `~/.zshrc`.
3. **Build tools and GLFW:** `brew install cmake ninja glfw`.

Homebrew can also supply the whole toolchain instead of the SDK. This is the configuration the course was written on:

```
brew install molten-vk vulkan-loader vulkan-headers vulkan-validationlayers \
  vulkan-tools shaderc glslang spirv-tools glfw cmake ninja
```

The manifest of Homebrew's validation layer names the layer's library without a directory, and the loader cannot find it, so creating an instance with validation fails with `VK_ERROR_LAYER_NOT_PRESENT`. Setting `DYLD_LIBRARY_PATH` to the layer's directory works in an interactive shell, but macOS removes `DYLD_` variables whenever a protected system program such as `/bin/sh` starts, so it fails as soon as a script runs your program. A fix that also works from scripts is a copy of the manifest that names the library by its full path, with `VK_LAYER_PATH` pointing at it:

```
layers=$(brew --prefix vulkan-validationlayers)
mkdir -p "$HOME/.local/share/vulkan-layers"
sed "s|\"library_path\": \"|\"library_path\": \"$layers/lib/|" \
  "$layers/share/vulkan/explicit_layer.d/VkLayer_khronos_validation.json" \
  > "$HOME/.local/share/vulkan-layers/VkLayer_khronos_validation.json"
export VK_LAYER_PATH="$HOME/.local/share/vulkan-layers"   # add this line to ~/.zshrc
```

On every Mac, MoltenVK differs from a native driver in several ways. It is a portability implementation, so programs must enable portability enumeration and the portability subset (Chapter 1.1). It has no `shaderFloat64` and no pipeline statistics queries. It translates shaders to Metal, which keeps its own shader cache (Chapter 1.4) and rejects names that collide with Metal's standard library. RenderDoc does not run on macOS, so use Xcode's Metal debugger instead (Chapter 1.6).

Check the installation

Run `vulkaninfo --summary`. It should list your GPU with an API version of 1.3 or later. On macOS it should name MoltenVK as the driver. If it reports no devices, the driver is missing or, on macOS, the SDK's environment is not set up in this shell.

The companion code

The code is `Vulkan-Code.zip`, a free download on the course page. It unpacks into `vulkan/code`; the volumes call this folder `code/`, and every command in them runs from inside it.

```
cd vulkan/code
cmake -S . -B build -G Ninja
cmake --build build
```

The build compiles the helper library `vkf`, its self-test, and every project of every unit, with each project's shaders compiled to SPIR-V beside it; the programs land in `build/bin`. To build only some units, configure with `-DVKF_UNITS="u1;u2"`. If CMake cannot find GLFW, it skips Unit 4's windowed projects with a message. Then run the self-test:

`vkf_selftest` on an Apple M2 Pro: a compute kernel checked on the CPU, an image written to a PNG file, and a dispatch on a second queue

```
$ ./build/bin/vkf_selftest
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
queues: main 0.0, compute 1.0, transfer 2.0
scale: 1048576 floats in 0.548 ms (15.3 GB/s)
gradient: wrote selftest.png
async compute: dispatch on queue family 1 signalled the timeline to 6
selftest passed
```

The first two lines name the device and driver `vkf` chose and confirm that validation and synchronisation validation are on. The last line must read `selftest passed`, and the program must exit with status 0.

Every `vkf` program reads four environment variables:

Variable	Effect
`VKF_VALIDATION=0`	Runs without the validation layer, for timing. Every measurement in the course uses it.
`VKF_SYNC_VALIDATION=0`	Keeps the validation layer but turns off synchronisation validation, which is the slowest part.
`VKF_DEVICE=<n>`	Uses device `n` in the order `vulkaninfo` lists them, instead of the best usable one, a discrete GPU before an integrated one.
`VKF_QUIET=1`	Suppresses the two start-up lines.

Unit 1′s programs use the raw API instead of `vkf`, and read only `VKF_VALIDATION`.

Validation layer versions

The course was written with version 1.4.363 of the validation layer. Older layers ignore settings they do not know, and their synchronisation validation may find less. In particular, the code enables a setting called `syncval_shader_accesses_heuristic`, without which the layer used for this course missed hazards between dispatches. If an older layer stays silent about a hazard the text says it reports, update the SDK.

Troubleshooting

Symptom	Cause and fix
`vkCreateInstance` fails with `VK_ERROR_INCOMPATIBLE_DRIVER`	No Vulkan driver was found. Install or update the GPU driver. On macOS, the program must enable portability enumeration, which the course's code does, and MoltenVK must be installed.
`VK_ERROR_LAYER_NOT_PRESENT`	The validation layer's manifest is installed but its library cannot be loaded. With Homebrew on macOS, apply the manifest fix above; elsewhere, reinstall the SDK. A layer that is not installed at all produces a message instead, and the program runs without validation.
`no device supports Vulkan 1.3`, or `VKF_DEVICE ... is not a usable device`	The driver is too old, or the GPU predates Vulkan 1.3. Update the driver, or choose another device with `VKF_DEVICE`.
CMake cannot find `glslc` or Vulkan	The SDK's environment is not set in this shell. Run its `setup-env.sh`, or on Windows reopen the prompt after installing.
Unit 4′s windowed targets are missing	CMake did not find GLFW. Install it and configure again; on Windows pass vcpkg's toolchain file.
Only `u4_glport` is missing	CMake did not find `GL/glcorearb.h`. Install the OpenGL headers as above; macOS needs nothing.
A pipeline fails to compile only on macOS	A name in a shader collides with Metal's standard library. Rename it (Chapter 1.4).
Validation messages you did not expect	Read them, as Chapter 1.6 shows. Programs exit with a non-zero status when the layer reports an error.

PART 6

Reference library

Each chapter's reading list names the sections to read with it. This part collects the sources in one place. Everything here is free unless it is marked as a book, and the links are to the official homes of the documents.

Specifications

- **The Vulkan specification** (Khronos), at docs.vulkan.org/spec, with every published version at registry.khronos.org/vulkan. It defines what is valid. Learn to search it by VUID (Chapter 1.6), and read the valid-usage list of every function you call for the first time.
- **The SPIR-V specification** (Khronos), at registry.khronos.org/SPIR-V, with the GLSL.std.450 instruction set.
- **The OpenGL Shading Language 4.60 specification** and the `GL_KHR_vulkan_glsl` **extension** that adapts it to Vulkan, in the Khronos OpenGL registry and the KhronosGroup/GLSL repository on GitHub.
- **The OpenGL 4.6 core profile specification** (Khronos), for the comparisons, at registry.khronos.org/OpenGL.

Guides and samples

- **The Vulkan Guide** (Khronos), at docs.vulkan.org/guide: short, authoritative chapters on topics the specification spreads out, from the loader to synchronisation. Its "Vulkan Decoder Ring" maps Vulkan terms to those of other APIs.
- **Vulkan Samples** (Khronos), at github.com/KhronosGroup/Vulkan-Samples: runnable samples of API features and performance practices, each with an explanation.
- **Sascha Willems' Vulkan examples**, at github.com/SaschaWillems/Vulkan: compact examples of nearly every rendering and compute technique, kept current for many years.
- **The Vulkan hardware database** by Sascha Willems, at vulkan.gpuinfo.org: the reported features, limits and formats of thousands of devices and drivers. Check it before relying on an optional feature.

Synchronisation and render graphs

- Hans-Kristian Arntzen, "Yet another blog explaining Vulkan synchronization" (2019), on his blog, themaister.net. The clearest informal account of the model that Chapter 3.2 teaches.
- The "Synchronization Examples" in the Vulkan Guide, a catalogue of common cases with the barriers each needs.
- Yuriy O'Donnell, "FrameGraph: Extensible Rendering Architecture in Frostbite", GDC 2017. The talk that introduced render graphs to most of the industry.
- Hans-Kristian Arntzen, "Render graphs and Vulkan: a deep dive" (2017), on themaister.net, which works through barrier derivation and aliasing in a real renderer.

Memory

- Adam Sawicki, "Memory management in Vulkan and DX12", GDC 2018.
- The documentation of AMD's Vulkan Memory Allocator, in particular "Choosing memory type" and "Recommended usage patterns".

GPU computing

- Wen-mei W. Hwu, David B. Kirk and Izzat El Hajj, *Programming Massively Parallel Processors*, 4th edition, Morgan Kaufmann, 2022 (book). Written for CUDA, but its chapters on the execution model, memory, reduction, scan, histograms, sorting and stencils apply directly to Unit 2.
- Mark Harris, "Optimizing Parallel Reduction in CUDA", NVIDIA, 2007. The classic sequence of reduction kernels that Chapter 2.3 follows.
- Mark Harris, Shubhabrata Sengupta and John D. Owens, "Parallel Prefix Sum (Scan) with CUDA", *GPU Gems 3*, chapter 39, 2007.
- Duane Merrill and Michael Garland, "Single-pass Parallel Prefix Scan with Decoupled Look-back", NVIDIA technical report, 2016.
- Lars Nyland, Mark Harris and Jan Prins, "Fast N-Body Simulation with CUDA", *GPU Gems 3*, chapter 31, 2007.
- Samuel Williams, Andrew Waterman and David Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures", *Communications of the ACM* 52(4), 2009.

Ray tracing

- Peter Shirley, Trevor David Black and Steve Hollasch, the *Ray Tracing in One Weekend* series, at raytracing.github.io. The best short introduction to the path tracing that Chapter 2.7 runs on the GPU.
- Matt Pharr, Wenzel Jakob and Greg Humphreys, *Physically Based Rendering: From Theory to Implementation*, 4th edition, MIT Press, 2023 (book, also free at pbr-book.org). The reference for everything from bounding volume hierarchies to materials.

Books on Vulkan

- Graham Sellers with John Kessenich, *Vulkan Programming Guide*, Addison-Wesley, 2016. Thorough on Vulkan 1.0; read it for the original design and check newer practice against the specification.
- Marco Castorina and Gabriel Sassone, *Mastering Graphics Programming with Vulkan*, Packt, 2023. A modern renderer, including a frame graph, multithreaded recording and GPU-driven rendering.

Vendor guides

- NVIDIA, "Vulkan Dos and Don'ts", on the NVIDIA Developer blog.
- AMD, the *RDNA Performance Guide*, on GPUOpen.
- Arm, the *Mali GPU Best Practices Developer Guide*, for tile-based GPUs.
- Apple, the *Metal Feature Set Tables*, for the limits MoltenVK inherits on each Apple GPU.

Tools

- **RenderDoc**, at renderdoc.org: frame capture and inspection on Windows and Linux.
- **NVIDIA Nsight Graphics**, **AMD Radeon GPU Profiler** and **Intel Graphics Performance Analyzers**: vendor profilers with hardware counters.
- **Xcode's Metal debugger**, for captures on macOS through MoltenVK.
- **The Vulkan Configurator**, `vkconfig`, in the SDK: layer settings for any program.

PART 7

Progress tracker

This list has every lab in the course, in order. Date each line when the lab's done-when criteria all hold and its evidence is filed, and the chapter line when its completion checklist is ticked. The sign-off page at the end of each unit is the formal record, and this list shows how far you have got.

Unit 1: The Explicit API

Lab	Title	Date done
Chapter 1.1 Instances, devices and queues		
Lab 1.1.1	Run the device report	
Lab 1.1.2	Break the instance on purpose	
Lab 1.1.3	Read more of the feature chain	
Lab 1.1.4	Ask for something the device lacks	
Checklist	*Every line of Chapter 1.1's completion checklist ticked*	
Chapter 1.2 Memory, buffers and images		
Lab 1.2.1	Map your machine's memory	
Lab 1.2.2	Round trips and coherence	
Lab 1.2.3	A thousand buffers in one allocation	
Lab 1.2.4	Survey formats and image memory	
Checklist	*Every line of Chapter 1.2's completion checklist ticked*	
Chapter 1.3 Commands and submission		
Lab 1.3.1	Fill, update, copy and read back	
Lab 1.3.2	Remove each barrier in turn	
Lab 1.3.3	Staging against direct writes	
Lab 1.3.4	What a submission costs	
Checklist	*Every line of Chapter 1.3's completion checklist ticked*	
Chapter 1.4 Shaders, SPIR-V and pipelines		
Lab 1.4.1	Compile, validate, disassemble	
Lab 1.4.2	Break the pipeline on purpose	
Lab 1.4.3	Two specialisation constants	
Lab 1.4.4	Warm and cold caches	
Checklist	*Every line of Chapter 1.4's completion checklist ticked*	
Chapter 1.5 Descriptors and your first dispatch		
Lab 1.5.1	SAXPY end to end	
Lab 1.5.2	Vector addition	
Lab 1.5.3	Ping-pong between two sets	
Lab 1.5.4	`std430` by hand	
Checklist	*Every line of Chapter 1.5's completion checklist ticked*	
Chapter 1.6 Validation, debugging and a helper layer		
Lab 1.6.1	Read six broken programs	
Lab 1.6.2	Print from a shader	
Lab 1.6.3	Capture a dispatch	
Lab 1.6.4	Your own RAII layer	

Lab	Title	Date done
Checklist	*Every line of Chapter 1.6's completion checklist ticked*	

Unit 2: Compute Shaders

Lab	Title	Date done
Chapter 2.1 The compute execution model		
Lab 2.1.1	Map a dispatch	
Lab 2.1.2	Three dimensions	
Lab 2.1.3	Sweep the local size on your GPU	
Lab 2.1.4	Exceed the limits	
Checklist	*Every line of Chapter 2.1's completion checklist ticked*	
Chapter 2.2 Memory in shaders		
Lab 2.2.1	Transpose at four sizes	
Lab 2.2.2	Read your bank conflicts	
Lab 2.2.3	Histograms under contention	
Lab 2.2.4	Races, observed and fixed	
Checklist	*Every line of Chapter 2.2's completion checklist ticked*	
Chapter 2.3 Parallel patterns		
Lab 2.3.1	Reduce at the bandwidth limit	
Lab 2.3.2	A scan by hand, then in code	
Lab 2.3.3	Compaction that returns indices	
Lab 2.3.4	Sort with eight-bit digits	
Checklist	*Every line of Chapter 2.3's completion checklist ticked*	
Chapter 2.4 Images and image processing		
Lab 2.4.1	Blur at several radii	
Lab 2.4.2	Two taps per fetch	
Lab 2.4.3	Blur into 8-bit storage	
Lab 2.4.4	Divergence and the shape of the workgroup	
Checklist	*Every line of Chapter 2.4's completion checklist ticked*	
Chapter 2.5 Measuring and tuning		
Lab 2.5.1	Your GPU's roofline	
Lab 2.5.2	Timestamps by hand	
Lab 2.5.3	Tune on your GPU	
Lab 2.5.4	Look inside with a profiler	
Checklist	*Every line of Chapter 2.5's completion checklist ticked*	
Chapter 2.6 Dispatch at scale		
Lab 2.6.1	Run, scale and check	
Lab 2.6.2	Without tiling	
Lab 2.6.3	Change the particle count on the GPU	
Lab 2.6.4	Addresses in data	
Checklist	*Every line of Chapter 2.6's completion checklist ticked*	
Chapter 2.7 A ray tracer in a compute shader		
Lab 2.7.1	Measure convergence	
Lab 2.7.2	Break it on purpose	
Lab 2.7.3	Size the stack	
Lab 2.7.4	Measure the divergence	
Checklist	*Every line of Chapter 2.7's completion checklist ticked*	

Unit 3: Synchronisation

Lab	Title	Date done
Chapter 3.1 Why synchronisation is your job		
Lab 3.1.1	Run the catalogue	
Lab 3.1.2	Try to make a hazard show	
Lab 3.1.3	Write the barriers yourself	
Checklist	*Every line of Chapter 3.1's completion checklist ticked*	
Chapter 3.2 The dependency model		
Lab 3.2.1	Barriers for a table of scenarios	
Lab 3.2.2	Too narrow and too wide	
Lab 3.2.3	Split a dependency into a chain	
Lab 3.2.4	Layouts by hand	
Checklist	*Every line of Chapter 3.2's completion checklist ticked*	
Chapter 3.3 Pipeline barriers in practice		
Lab 3.3.1	Justify every barrier	
Lab 3.3.2	Measure barriers on your GPU	
Lab 3.3.3	Independent work across a barrier	
Lab 3.3.4	Batch and unbatch	
Checklist	*Every line of Chapter 3.3's completion checklist ticked*	
Chapter 3.4 Fences, semaphores and events		
Lab 3.4.1	Two timelines	
Lab 3.4.2	Wait before signal	
Lab 3.4.3	Where events pay	
Lab 3.4.4	What idling costs	
Checklist	*Every line of Chapter 3.4's completion checklist ticked*	
Chapter 3.5 Queues and async compute		
Lab 3.5.1	Map your queues	
Lab 3.5.2	Hand over an image	
Lab 3.5.3	Find the overlap	
Lab 3.5.4	Break the transfer	
Checklist	*Every line of Chapter 3.5's completion checklist ticked*	
Chapter 3.6 Frames in flight		
Lab 3.6.1	Who limits the frame	
Lab 3.6.2	Break the ring	
Lab 3.6.3	Destroy too soon	
Lab 3.6.4	Measure the latency	
Checklist	*Every line of Chapter 3.6's completion checklist ticked*	
Chapter 3.7 Finding synchronisation bugs		
Lab 3.7.1	Fix the bugs from their reports	
Lab 3.7.2	An atomic blind spot	
Lab 3.7.3	From report to barrier	
Lab 3.7.4	The bugs on a second device	
Checklist	*Every line of Chapter 3.7's completion checklist ticked*	
Chapter 3.8 Render graphs		
Lab 3.8.1	Add a pass	
Lab 3.8.2	Memory aliasing	
Lab 3.8.3	From one frame to the next	
Lab 3.8.4	Schedule for overlap	

Lab	Title	Date done
Checklist	*Every line of Chapter 3.8's completion checklist ticked*	

Unit 4: Rendering and the Whole Frame

Lab	Title	Date done
Chapter 4.1 The graphics pipeline		
Lab 4.1.1	Read the triangle's pixels	
Lab 4.1.2	State you can change	
Lab 4.1.3	Break the depth test	
Lab 4.1.4	Render passes and dynamic rendering	
Checklist	*Every line of Chapter 4.1's completion checklist ticked*	
Chapter 4.2 Presentation		
Lab 4.2.1	In a window and headless	
Lab 4.2.2	Present modes, image counts and latency	
Lab 4.2.3	Break the semaphores	
Lab 4.2.4	Resize without stalling	
Checklist	*Every line of Chapter 4.2's completion checklist ticked*	
Chapter 4.3 Drawing with data		
Lab 4.3.1	The scene with and without its features	
Lab 4.3.2	Mipmaps in the wrong colour space	
Lab 4.3.3	Anisotropy and level of detail	
Lab 4.3.4	A larger table	
Checklist	*Every line of Chapter 4.3's completion checklist ticked*	
Chapter 4.4 Compute meets graphics		
Lab 4.4.1	Run and time the frame	
Lab 4.4.2	Remove each barrier	
Lab 4.4.3	Cull more finely	
Lab 4.4.4	Exposure and the HDR image	
Checklist	*Every line of Chapter 4.4's completion checklist ticked*	
Chapter 4.5 From OpenGL to Vulkan		
Lab 4.5.1	Measure both APIs	
Lab 4.5.2	Instancing in both	
Lab 4.5.3	Clip space without the matrix	
Lab 4.5.4	Port a compute frame	
Checklist	*Every line of Chapter 4.5's completion checklist ticked*	
Chapter 4.6 Scaling up		
Lab 4.6.1	Scale the recording	
Lab 4.6.2	A pipeline cache across runs	
Lab 4.6.3	Account for memory	
Lab 4.6.4	Profile a frame	
Checklist	*Every line of Chapter 4.6's completion checklist ticked*	
Chapter 4.7 Capstone: a GPU-driven sandbox		
Lab 4.7.1	Run every configuration	
Lab 4.7.2	Move the cull	
Lab 4.7.3	A glow pass	
Lab 4.7.4	The final project	
Checklist	*Every line of Chapter 4.7's completion checklist ticked*	